

E2 – Thesis → Quant → Risk/Scenario Path

(Architecture & Testing Spec)

A. E2-Requirements

E2's job is to turn a **Thesis** with `expression_legs` into a **deterministic quant evaluation**, independent of any LLM or UI.

Concretely, E2 must:

1. **Provide a ThesisEvaluationService**

- Module: `src/slice/evaluation/thesis_evaluation.py`
- Public API:

```
class ThesisEvaluationService:
    def __init__(self, price_source):
        ...

    def evaluate_thesis(self, thesis: Thesis) -> ThesisEvaluationResult:
        ...
```

- 1.

-
- Deterministic: same thesis + same price data → same result, no randomness.

2. **Interpret `expression_legs` as a static allocation strategy**

- Use existing `Thesis.expression_legs`, each with:
 - `asset`: str
 - `direction`: "long" | "short"
 - `size_pct`: float (0–100)
- **Validation:**
 - Sum of `size_pct` across legs ≤ 100 .
 - Each leg `direction` $\in \{"long", "short"\}$.
 - Each asset has valid historical price data in `price_source`.
- Any invalid thesis must produce a clear error (e.g. `ValueError`), not silent failure.

3. **Run a static-allocation, daily-rebalanced backtest**

- Assume daily frequency.
- Portfolio weights:
 - $\text{weight}_i = \text{size_pct}_i / 100 * (1 \text{ or } -1)$ depending on direction.
- Daily portfolio return:
 - $r_{\text{portfolio},t} = \sum_i \text{weight}_i * r_{i,t}$

- Equity curve:
 - Start at 1.0 (normalized capital).
 - $\text{equity}_t = \text{equity}_{t-1} * (1 + r_{\text{portfolio},t})$.
 - Rebalance implicitly by using constant weights each day.
4. **Produce standard performance metrics**

Populate a ThesisEvaluationResult with at least:

- $\text{total_return} = (\text{final_value} / \text{initial_value}) - 1$
- $\text{cagr} = \text{annualized using years} = N_{\text{days}} / 252$
 - $\text{cagr} = (\text{final_value} / \text{initial_value})^{1/\text{years}} - 1$ if years > 0
- volatility: annualized std dev of daily returns
 - $\text{vol_daily} = \text{std}(\text{daily_returns})$
 - $\text{vol_annual} = \text{vol_daily} * \text{sqrt}(252)$
- sharpe: $(\text{cagr} - r_f) / \text{volatility}$ with $r_f \approx 0$
- max_drawdown : largest peak-to-trough % loss on equity curve

These must be numeric and well-defined (no NaNs in output).

5. **Output an equity curve timeseries**
- A list of {date, value} points, one per trading day in the backtest window.
 - This will be used by UI + LLM to inspect path behavior.
6. **Output basic risk metrics**
- At minimum:
 - **Concentration**: max absolute weight across legs (e.g. max_weight_pct).
 - **Tail risk**: a simple VaR-like metric or worst daily return:
 - e.g. $\text{VaR}_{95} = -\text{quantile}(\text{daily_returns}, 0.05)$ as a positive number.
 - **Max drawdown** reused as a risk metric.
 - Provide as risk_metrics: dict[str, float] in ThesisEvaluationResult.
7. **Perform scenario analysis**
- Implement a simple scenario engine (can be inline or separate class) that:
 - Holds a small set of predefined shocks (e.g. “All -10%”, “All +10%”).
 - Computes portfolio % P&L under each shock (pnl_pct , pnl_abs).
 - Output as scenarios: list[ScenarioImpact] in ThesisEvaluationResult.
8. **Validate and fail fast**
- If thesis is not evaluable (weights > 100%, no legs, missing price data, invalid direction):
 - Raise an exception (e.g. ValueError) with a clear message.
 - Only valid, fully evaluable theses proceed to quant.
9. **Be independent from LLM / UI**
- ThesisEvaluationService must not:
 - Call any LLM.
 - Depend on UI or HTTP.
 - Pure Python component usable directly by E4 (orchestrator) and E3 (LLM critique via DTO).
10. **Be testable with synthetic price data**

- All logic must work against a fake PriceSource returning synthetic pd.Series so tests are fully deterministic and offline.
-

B. E2-Research Summary

This section explains the quantitative choices and how they map to the design.

1. Backtesting approach

- We are implementing a **constant-mix portfolio**:
 - Expression legs define **fixed weights** for each asset.
 - Portfolio is effectively rebalanced daily to those weights.
- Portfolio return each day:
 - $r_{\text{portfolio},t} = \sum_i w_i * r_{i,t}$
 - Equivalent to applying a vector of weights to a matrix of returns.
- This is standard practice for simple allocation strategies and easy to implement in pandas.

2. Performance metrics

- **Total Return**:
 - $\text{TotalReturn} = (\text{Final} / \text{Initial}) - 1$
- **CAGR**:
 - $T_{\text{years}} = N_{\text{days}} / 252$
 - $\text{CAGR} = (\text{Final} / \text{Initial})^{(1/T_{\text{years}})} - 1$
- **Volatility**:
 - $\sigma_{\text{daily}} = \text{std}(\text{daily_returns})$
 - $\sigma_{\text{annual}} = \sigma_{\text{daily}} * \text{sqrt}(252)$
- **Sharpe Ratio**:
 - With $r_f \approx 0$, use $\text{Sharpe} = \text{CAGR} / \sigma_{\text{annual}}$.
 - For short periods, this can be noisy; this is an accepted caveat.
- **Max Drawdown**:
 - Compute running max of equity curve.
 - For each day: $\text{dd}_t = (\text{equity}_t - \text{max_to_date}) / \text{max_to_date}$.
 - Max drawdown = most negative of those (reported as positive magnitude).

3. Risk metrics

- **Concentration**:
 - Largest absolute weight across legs (max_weight_pct).
 - Flags overly concentrated theses (e.g. 80–100% in a single leg).
- **VaR-like metric**:
 - Historical one-day 95% VaR approximation:
 - $\text{VaR}_{95} = -\text{quantile}(\text{daily_returns}, 0.05)$ (positive number).
 - Simple but standard for high-level risk sense.
- **Max drawdown** reused:
 - Already computed for performance; also recorded in `risk_metrics`.

4. Scenario analysis

- Stress testing pattern:
 - Define deterministic shocks to prices (e.g. -10%, +10%) per asset.
 - Compute portfolio impact given current weights.
- Minimal demonstration:
 - “All -10%”: all assets shocked -10%.
 - For fully invested long-only, portfolio loses ~10%.
 - “All +10%”: all assets shocked +10%.
 - Gains ~10% for fully invested long-only.
- This is a simplified but standard template.

5. Libraries vs custom engine

- Existing backtest libraries (Backtrader, vectorbt, etc.) could handle this.
 - For E2 evaluation:
 - A **lightweight custom implementation** with pandas is simpler to control, easier to test, and avoids heavy dependency overhead.
 - PriceSource abstraction decouples E2 from any specific data store/API.
-

C. E2-Design

C1. PriceSource Interface

Introduce a PriceSource interface used by ThesisEvaluationService and, later, by E1’s portfolio functions for consistency.

Suggested interface:

```
# src/slice/quant/price_source.py

from datetime import date
import pandas as pd
from typing import Protocol

class PriceSource(Protocol):
    def get_history(self, asset: str, start: date, end: date) -> pd.Series:
        """
        Returns daily price series for `asset` between start and end.
        Index: datetime/date, Values: float prices.
        """
        ...
```

```
def get_current_price(self, asset: str) -> float:
    """
    Returns latest price for `asset`.
    """
    ...
```

Implementation details (actual concrete class) can vary; for E2, tests will use a dummy implementation.

C2. DTO Models

Define DTOs in a dedicated module, e.g. `src/slice/models/evaluation.py`:

```
# src/slice/models/evaluation.py

from datetime import datetime
from pydantic import BaseModel
from typing import List, Dict

class EquityPoint(BaseModel):
    date: datetime
    value: float

class ScenarioImpact(BaseModel):
    name: str
    pnl_abs: float    # absolute P&L in portfolio value units
    pnl_pct: float    # P&L as % of portfolio value (e.g. -10 = -10%)

class ThesisEvaluationResult(BaseModel):
    performance: Dict[str, float]    # e.g. {"total_return": 10.0, ...}
    timeseries: List[EquityPoint]    # equity curve
    risk_metrics: Dict[str, float]    # e.g. {"max_weight_pct": 80.0, ...}
    scenarios: List[ScenarioImpact]
```

Conventions:

- **Performance** values in **percentage units** (10.0 means 10%).
 - `max_drawdown` and `VaR_95` also in % units (e.g. 20.0 is 20%).
 - `timeseries` starts at 1.0 for the first date (normalized capital).
-

C3. ThesisEvaluationService

Module:

- `src/slice/evaluation/thesis_evaluation.py`

Public signature:

```
from slice.models.thesis import Thesis
from slice.models.evaluation import ThesisEvaluationResult
from slice.quant.price_source import PriceSource

class ThesisEvaluationService:
    def __init__(self, price_source: PriceSource):
        self.price_source = price_source

    def evaluate_thesis(self, thesis: Thesis) -> ThesisEvaluationResult:
        ...
```

C3.1 Internal Steps

`evaluate_thesis` executes the following pipeline:

1. **Validate thesis expression**
 - Check:
 - `thesis.expression_legs` is non-empty.
 - direction in {"long", "short"}.
 - `sum(size_pct) <= 100`.
 - On violation: raise `ValueError` with specific message.
2. **Determine backtest period**

Strategy:

- If thesis has start/end dates, use them.
 - Else, pick a reasonable default, e.g.:
 - `start_date = today - 2 years`
 - `end_date = today`
 - For tests, the `PriceSource` will fix the dates anyway.
3. **Fetch historical prices and align**
 - For each `leg.asset`:
 - `series = price_source.get_history(asset, start_date, end_date)`
 - Validate non-empty; else raise `ValueError("No price data for asset X")`.
 - Combine into `DataFrame`:

```
prices_df = pd.DataFrame(asset_prices) # columns = asset tickers
prices_df.dropna(inplace=True) # strict: require all assets present on each
day
```

3.
 -

- If resulting DataFrame empty → raise ValueError("No overlapping price history for thesis assets").

4. Compute daily returns and weights

- Daily returns:

```
daily_returns = prices_df.pct_change().fillna(0.0)
```

4.

-
- Weights dict:

```
weights = {
    leg.asset: (leg.size_pct / 100.0) * (1.0 if leg.direction == "long" else
-1.0)
    for leg in thesis.expression_legs
}
```

4.

-
- We **do not re-normalize** if $\text{sum}(\text{abs}(\text{weights})) < 1$:
 - Unallocated capital assumed to sit in cash at 0% return.

5. Compute daily portfolio returns

- Convert weights to aligned pd.Series and compute:

```
w_series = pd.Series(weights)
daily_portfolio_returns = daily_returns.mul(w_series, axis=1).sum(axis=1)
```

5.

-
- This yields one portfolio return per day.

6. Build equity curve

- Start at 1.0:

```
equity_curve = (1.0 + daily_portfolio_returns).cumprod()
```

6.

-
- Convert to list of EquityPoint:

```
equity_points = [
    EquityPoint(date=idx.to_pydatetime(), value=float(val))
    for idx, val in equity_curve.items()
]
```

6.

7. Compute performance metrics

Let:

- `initial_value = equity_curve.iloc[0]` (should be 1.0).
- `final_value = equity_curve.iloc[-1]`.
- `N_days = len(equity_curve)`.

Metrics:

- `total_return_frac = final_value / initial_value - 1.0`
- `years = N_days / 252.0` (if years > 0, otherwise treat as 0).
- `cagr_frac = (final_value / initial_value) ** (1.0 / years) - 1.0` if years > 0 else 0.0.
- `vol_daily = daily_portfolio_returns.std()` (0 if < 2 observations).
- `vol_annual = vol_daily * (252.0 ** 0.5)`
- `sharpe = cagr_frac / vol_annual` if vol_annual > 0, else 0.0.
- Max drawdown:

```
running_max = equity_curve.cummax()
drawdowns = (equity_curve - running_max) / running_max
max_drawdown_frac = -drawdowns.min() # positive fraction
```

7.

Convert metrics to percentages:

```
performance = {
    "total_return": total_return_frac * 100.0,
    "cagr": cagr_frac * 100.0,
    "volatility": vol_annual * 100.0,
    "sharpe": sharpe,
    "max_drawdown": max_drawdown_frac * 100.0,
}
```

7.

8. Compute risk metrics

- Concentration:

```
max_weight_abs = max(abs(w) for w in weights.values()) if weights else 0.0
```

8.

-
- Historical VaR (95%):

```
var_95 = daily_portfolio_returns.quantile(0.05) if
len(daily_portfolio_returns) > 0 else 0.0
VaR_95_pct = -var_95 * 100.0
```

8.

-
- Use `max_drawdown_frac` from performance.
- Build dict:

```
risk_metrics = {
    "max_weight_pct": max_weight_abs * 100.0,
    "VaR_95": VaR_95_pct,
    "max_drawdown_pct": max_drawdown_frac * 100.0,
}
```

8.

9. Compute scenarios

Minimal implementation:

- Get last prices:

```
current_prices = prices_df.iloc[-1].to_dict() # asset -> price
```

9.

-
- Helper to compute portfolio P&L given new prices:

```
def scenario_pnl_pct(new_prices: dict[str, float]) -> float:
    pnl_frac = 0.0
    for asset, old_price in current_prices.items():
        new_price = new_prices[asset]
        asset_return = (new_price - old_price) / old_price
        pnl_frac += weights[asset] * asset_return
    return pnl_frac
```

9.

-
- Define scenarios:
 - **All -10%:**

```
new_prices = {a: p * 0.9 for a, p in current_prices.items()}
pnl_frac = scenario_pnl_pct(new_prices)
scenarios.append(
    ScenarioImpact(
        name="All -10%",
        pnl_pct=pnl_frac * 100.0,
        pnl_abs=pnl_frac * float(equity_curve.iloc[-1]),
    )
)
```

9.

-
-
- **All +10%:**

```
new_prices = {a: p * 1.1 for a, p in current_prices.items()}
pnl_frac = scenario_pnl_pct(new_prices)
scenarios.append(
    ScenarioImpact(
```

```

        name="All +10%",
        pnl_pct=pnl_frac * 100.0,
        pnl_abs=pnl_frac * float(equity_curve.iloc[-1]),
    )
)

```

9.

-
- Additional scenarios (e.g. asset-class-specific) can be added later, but E2 requires at least these two.

10. Construct and return ThesisEvaluationResult

```

result = ThesisEvaluationResult(
    performance=performance,
    timeseries=equity_points,
    risk_metrics=risk_metrics,
    scenarios=scenarios,
)
return result

```

C4. Integration Notes

- E2 is **pure quant**:
 - Called by orchestrator (E4) when doing a thesis evaluation session.
 - Its ThesisEvaluationResult will be passed into:
 - LLM tools in E3 (for critique).
 - UI in E6 (for displaying metrics/scenarios/equity curve).
- No DB writes in E2:
 - Persistence of evaluation results, if needed, happens in E4.

D. E2-Testing Plan

All tests use **synthetic price data** via a fake PriceSource.

D1. PriceSource Stub

For tests, implement:

```
class FakePriceSource:
```

```

def __init__(self, series_by_asset: dict[str, pd.Series]):
    self.series_by_asset = series_by_asset

def get_history(self, asset: str, start: date, end: date) -> pd.Series:
    return self.series_by_asset.get(asset, pd.Series(dtype=float))

def get_current_price(self, asset: str) -> float:
    series = self.series_by_asset[asset]
    return float(series.iloc[-1])

```

D2. Test Cases

1. Single-asset, monotonic up

- Asset A: prices 100 $\rightarrow$ 110 over N days (e.g. linear or geometric).
- Thesis: 100% long A.
- Expectations:
 - $\text{total_return} \approx \text{asset's total return (\%)}$.
 - $\text{max_drawdown} \approx 0$ (if strictly monotonic).
 - $\text{risk_metrics["max_weight_pct"]} = 100$.
 - $\text{equity_curve length} = N$.
 - First point ≈ 1.0 , last $\approx (1 + \text{total_return_frac})$.

2. Single-asset, monotonic down

- Asset B: prices 100 $\rightarrow$ 90.
- Thesis: 100% long B.
- Expectations:
 - total_return negative.
 - $\text{max_drawdown} \approx |\text{total_return}|$.
 - Sharpe negative.

3. Two assets, opposing trends

- A up, B down.
- Thesis: 50% long A, 50% long B.
- Expect roughly flat portfolio (if symmetric).
- Checks:
 - total_return near 0.
 - Equity curve shows some drawdown when B drag dominates.
 - $\text{max_weight_pct} = 50$.

4. Short position

- Asset A up.
- Thesis: 100% short A.
- Expectations:
 - $\text{total_return} \approx -(\text{A's total return})$.
 - $\text{max_weight_pct} = 100$.
 - Scenario "All -10%":
 - Short portfolio should profit; $\text{pnl_pct} \approx +10$.

5. Partial cash

- Thesis: 50% long A, no other legs.
- The rest 50% is implicit cash:

- Portfolio total $_return \approx 0.5 * \text{asset's total return}$.
 - Volatility $\sim 0.5 * \text{asset volatility}$.
6. **Scenario outputs**
- Use a simple case:
 - One asset A only, long 100%.
 - For “All -10%”:
 - Expect `scenarios[?].pnl_pct` around -10.
 - For “All +10%”:
 - Expect `scenarios[?].pnl_pct` around +10.
 - For mixed weights (e.g. 50/50 A/B):
 - Confirm `pnl_pct` matches $\sum w_i * \text{shock}$.
7. **Risk metrics correctness**
- Build a return path with known drawdown:
 - Equity: $1.0 \rightarrow 1.2 \rightarrow 0.9 \rightarrow 1.3$
 - Max drawdown = $(1.2 - 0.9) / 1.2 = \sim 25\%$.
 - Verify `risk_metrics["max_drawdown_pct"]  $\approx 25$` .
 - For VaR:
 - Construct daily returns with a known 5th percentile, e.g. set one day return = -0.02 and others small; check `VaR_95  $\approx 2$` .
8. **Validation / error cases**
- Legs sum > 100%:
 - Expect `ValueError` with message about allocation > 100%.
 - Empty `expression_legs`:
 - Expect `ValueError` about no legs.
 - Unknown direction:
 - Expect `ValueError` about invalid direction.
 - Missing price data:
 - `FakePriceSource` returns empty series $\rightarrow$ expect `ValueError("No price data for asset X")`.
-

E. E2-Risks & Dependencies

1. **Data availability and alignment**
 - Assumes all assets used in a thesis have sufficiently long, aligned daily histories.
 - Current implementation uses **inner join** on dates and drops any dates with NaNs.
 - Implications:
 - Multi-market assets with different holidays may lose many days.
 - For evaluation harness, acceptable; but must be documented.
2. **Annualization assumptions**
 - Uses 252 trading days/year.
 - If input data is not daily (e.g. weekly), metrics will be mis-scaled.
 - For Slice’s intended use (daily price data), assumption is reasonable.
3. **Short positions and leverage**
 - Short is modeled as negative weight with no costs or margin constraints.

- Extreme moves can push equity < 0 in theory.
 - The engine does not clamp at 0; negative equity simply indicates a blown-out strategy.
 - For evaluation, this is acceptable but should be noted.
 - 4. **Transaction costs and slippage**
 - Completely ignored.
 - Daily rebalancing with no friction overestimates realistic performance.
 - Acceptable for **relative** evaluation, not for production-grade P&L.
 - 5. **CAGR for short periods**
 - For very short test windows, CAGR can be numerically large and misleading.
 - For E2 tests, that is fine; real usage should use longer windows.
 - 6. **Output serialization**
 - Pydantic models are JSON-serializable.
 - Need to ensure:
 - No non-serializable values (e.g. inf).
 - In this spec we clamp sharpe to finite numbers (0 if vol=0).
 - 7. **Coupling with later phases**
 - E3 (LLM) and E4 (orchestrator) depend on:
 - The exact keys in performance and risk_metrics.
 - Any changes must be coordinated; this spec should be treated as contract.
-

F. E2 Completion Checklist

To claim E2 is **done**, all of the following must exist and pass tests:

1. **Code artifacts**
 - src/slice/quant/price_source.py
 - PriceSource interface with get_history, get_current_price.
 - src/slice/models/evaluation.py
 - EquityPoint, ScenarioImpact, ThesisEvaluationResult.
 - src/slice/evaluation/thesis_evaluation.py
 - ThesisEvaluationService.__init__(price_source)
 - ThesisEvaluationService.evaluate_thesis(thesis: Thesis) -> ThesisEvaluationResult
 - Implements:
 - Thesis validation (legs, sum, directions).
 - Price fetch + alignment.
 - Daily returns computation.
 - Equity curve.
 - Performance metrics.
 - Risk metrics.
 - Scenario analysis (“All -10%”, “All +10%”).

2. Testing

- Tests using a FakePriceSource:
 - Single-asset up/down cases.
 - Two-asset mixed case.
 - Short position case.
 - Partial-cash case.
 - Scenario correctness for at least “All -10%” and “All +10%”.
 - Risk metrics (max drawdown, VaR_95).
 - Error paths (invalid thesis, missing data).
- All tests passing in CI.

3. Contract stability

- ThesisEvaluationResult fields and key names are documented and stable:
 - performance contains: total_return, cagr, volatility, sharpe, max_drawdown.
 - risk_metrics contains: max_weight_pct, VaR_95, max_drawdown_pct.
 - scenarios is a list of ScenarioImpact with name, pnl_abs, pnl_pct.
- No dependence on any LLM or UI module.

If any item in this checklist is missing or not test-covered, E2 is not complete.